Payment & Subscription API — Frontend Integration Guide

Base URL: `{API_BASE}/api/v1`

---

1. Client Endpoints (Authenticated)

All client endpoints require `Authorization: Bearer {token}` header.

1.1 Subscription Checkout

```
POST /subscriptions/checkout
```

Initiates a payment checkout for a subscription package. Redirects the user to an external payment page.

Request:

```
{
  "package_slug": "starter",
  "payment_method": "edahabia"
}
```

Field	Type	Required	Notes
`package_slug`	string	yes	Must match an active package slug
`payment_method`	string	no	One of the currently enabled methods (e.g. `edahabia`, `cib`). If omitted, the default gateway is used.

Response (201):

```
{
  "success": true,
  "data": {
    "checkout_url": "https://pay.chargily.net/test/checkouts/...",
    "payment_id": 42
  }
}
```

If a pending payment already exists for the same package:

```
{
  "success": true,
  "data": {
    "checkout_url": "https://pay.chargily.net/test/checkouts/...",
    "payment_id": 38
  }
}
```

Frontend flow:

1. Call `POST /subscriptions/checkout` with package and (optional) payment method
2. Redirect the user to `checkout_url`
3. User completes payment on the gateway's hosted page
4. Gateway redirects user back to `success_url` or `failure_url` (configured server-side)
5. A webhook processes the result. Poll `GET /subscriptions/payments/{payment_id}` or `GET /subscriptions/current` to confirm activation.

1.2 Payment History

```
GET /subscriptions/payments?per_page=15
```

Paginated list of all payment records for the authenticated client.

Response:

```
{
  "success": true,
  "data": [
    {
      "id": 42,
      "client_id": 1,
      "package_id": 3,
      "gateway": "chargily",
      "amount": 5000,
      "currency": "dzd",
      "status": "paid",
      "payment_method": "edahabia",
      "checkout_url": "https://...",
      "created_at": "2026-02-01T12:00:00.000000Z",
      "package": { "id": 3, "name": "Starter", "slug": "starter" }
    }
  ],
  "pagination": { "current_page": 1, "per_page": 15, "total": 1,
"total_pages": 1 }
}
```

Payment `status` values: `pending`, `paid`, `failed`, `canceled`.

1.3 Payment Detail

```
GET /subscriptions/payments/{payment_id}
```

Returns full details of a single payment, including related package info.

Response:

```
{
  "success": true,
  "data": {
    "id": 42,
    "client_id": 1,
    "package_id": 3,
    "gateway": "chargily",
    "amount": 5000,
    "currency": "dzd",
    "status": "paid",
    "payment_method": "edahabia",
    "checkout_url": "https://...",
    "gateway_checkout_id": "chk_abc123",
    "paid_at": "2026-02-01T12:05:00.000000Z",
    "created_at": "2026-02-01T12:00:00.000000Z",
    "package": {
      "id": 3,
      "name": "Starter",
      "slug": "starter",
      "price": 5000,
      "currency": "dzd",
      "billing_cycle": "monthly"
    }
  }
}
```

---

2. Super Admin Endpoints

All super admin endpoints require `Authorization: Bearer {token}` with a super admin token.

Base prefix: `/api/v1/super-admin`

2.1 List All Payment Gateways

```
GET /super-admin/payment-gateways
```

Returns all configured gateways, current settings, and the payment method routing map.

Response:

```
{
  "success": true,
  "data": {
    "gateways": [
      {
        "identifier": "chargily",
        "name": "Chargily Pay",
        "is_enabled": true,
        "is_default": true,
        "has_credentials": true,
        "credentials_updated_at": "2026-01-15T10:00:00.000000Z",
        "credential_fields": [
          { "key": "secret_key", "label": "Secret Key", "type": "secret",
"required": true },
          { "key": "public_key", "label": "Public Key", "type": "string",
"required": true }
        ],
        "setting_fields": [
          { "key": "sandbox", "label": "Sandbox Mode", "type": "boolean",
"default": true },
          { "key": "locale", "label": "Locale", "type": "string",
"default": "fr", "options": ["fr", "en", "ar"] }
        ],
        "current_settings": { "sandbox": true, "locale": "fr" },
        "supported_currencies": ["dzd"],
        "driver_supported_methods": ["edahabia", "cib"],
        "assigned_payment_methods": ["edahabia", "cib"]
      },
      {
        "identifier": "slickpay",
        "name": "SlickPay",
        "is_enabled": true,
        "is_default": false,
        "has_credentials": false,
        "credential_fields": [
          { "key": "public_key", "label": "Public Key", "type": "string",
"required": true },
          { "key": "webhook_secret", "label": "Webhook Secret", "type":
"secret", "required": true }
        ],
        "setting_fields": [
          { "key": "sandbox", "label": "Sandbox Mode", "type": "boolean",
"default": true }
        ],
        "current_settings": { "sandbox": true },
        "supported_currencies": ["dzd"],
        "driver_supported_methods": ["edahabia"],
```

```
          "assigned_payment_methods": []
        }
      ],
      "default_gateway": "chargily",
      "payment_method_routing": {
        "edahabia": "chargily",
        "cib": "chargily"
      }
    }
  }
```

Key fields:

- `driver_supported_methods` — what the gateway driver *can* handle
- `assigned_payment_methods` — what is currently assigned to this gateway
- `payment_method_routing` — global map of which payment method routes to which gateway

---

2.2 Show Single Gateway

```
GET /super-admin/payment-gateways/{gateway}
```

Returns the same gateway object as in the index (single item).

2.3 Update Gateway Credentials

```
PUT /super-admin/payment-gateways/{gateway}/credentials
```

Request (Chargily example):

```
{
  "credentials": {
    "secret_key": "live_sk_...",
    "public_key": "live_pk_..."
  }
}
```

Request (SlickPay example):

```
{
  "credentials": {
    "public_key": "sp_pk_...",
    "webhook_secret": "whsec_..."
```

```
  }
}
```

Required fields come from `credential_fields` in the gateway data. Returns the updated gateway object.

2.4 Update Gateway Settings

```
PUT /super-admin/payment-gateways/{gateway}/settings
```

Request:

```
{
  "is_enabled": true,
  "settings": {
    "sandbox": false,
    "locale": "en"
  }
}
```

Both `is_enabled` and `settings` are optional. The default gateway cannot be disabled (returns 400).

2.5 Set Default Gateway

```
POST /super-admin/payment-gateways/{gateway}/set-default
```

No body required. A disabled gateway cannot be set as default (returns 400).

2.6 List Payment Methods for a Gateway

```
GET /super-admin/payment-gateways/{gateway}/payment-methods
```

Response:

```
{
  "success": true,
  "data": {
    "gateway": "chargily",
    "driver_supported_methods": ["edahabia", "cib"],
    "assigned_payment_methods": ["edahabia"]
```

```
    }
  }
```

2.7 Assign Payment Method to a Gateway

```
POST /super-admin/payment-gateways/{gateway}/payment-methods
```

Request:

```
{
  "payment_method": "cib"
}
```

Response (201):

```
{
  "success": true,
  "data": {
    "payment_method": "cib",
    "gateway": "chargily"
  }
}
```

Rules:

- `payment_method` must be in the gateway's `driver_supported_methods`
- Each payment method can only be assigned to **one** gateway globally (unique constraint)
- If the gateway has no DB record yet, one is auto-created

2.8 Remove Payment Method from a Gateway

```
DELETE /super-admin/payment-gateways/{gateway}/payment-
methods/{paymentMethod}
```

Response (200):

```
{ "success": true, "data": null }
```

Returns 404 if the method is not assigned to this gateway.

---

3. Error Responses

401 — Unauthenticated:

```
{ "success": false, "error": { "code": "UNAUTHENTICATED", "message": "..."
} }
```

404 — Not Found:

```
{ "success": false, "error": { "code": "NOT_FOUND", "message": "Payment
gateway not found." } }
```

422 — Validation Error:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "...",
    "details": {
      "payment_method": ["The selected payment method is invalid."]
    }
  }
}
```

400 — Business Rule Violation:

```
{ "success": false, "error": { "code": "CANNOT_DISABLE_DEFAULT",
"message": "Cannot disable the default payment gateway." } }
```

502 — Gateway Error (checkout failed):

```
{ "success": false, "error": { "code": "CHECKOUT_FAILED", "message":
"...", "details": { "reason": "..." } } }
```

---

4. Usage Examples (Axios)

```
const api = axios.create({
  baseURL: 'https://api.markium.com/api/v1',
```

```
  headers: { Authorization: `Bearer ${token}` },
});

// ── Client: Checkout ──────────────────────────
const { data } = await api.post('/subscriptions/checkout', {
  package_slug: 'starter',
  payment_method: 'edahabia',
});
window.location.href = data.data.checkout_url;

// ── Client: Check payment status ──────────────
const payment = await
api.get(`/subscriptions/payments/${data.data.payment_id}`);
console.log(payment.data.data.status); // "pending" | "paid" | "failed" |
"canceled"

// ── Client: Payment history ───────────────────
const history = await api.get('/subscriptions/payments');
```

```
const admin = axios.create({
  baseURL: 'https://api.markium.com/api/v1/super-admin',
  headers: { Authorization: `Bearer ${adminToken}` },
});

// ── Admin: List all gateways ──────────────────
const { data } = await admin.get('/payment-gateways');
const routing = data.data.payment_method_routing;

// ── Admin: Update credentials ─────────────────
await admin.put('/payment-gateways/chargily/credentials', {
  credentials: { secret_key: 'live_sk_...', public_key: 'live_pk_...' },
});

// ── Admin: Toggle sandbox off ─────────────────
await admin.put('/payment-gateways/chargily/settings', {
  settings: { sandbox: false },
});

// ── Admin: Assign payment method ──────────────
await admin.post('/payment-gateways/chargily/payment-methods', {
  payment_method: 'cib',
});

// ── Admin: Remove payment method ──────────────
await admin.delete('/payment-gateways/chargily/payment-methods/cib');

// ── Admin: Set default gateway ────────────────
await admin.post('/payment-gateways/slickpay/set-default');
```

---

5. Payment Method Routing — How It Works

Each payment method (e.g. `edahabia`, `cib`) is assigned to exactly one gateway. When a client submits a checkout with `payment_method: "cib"`, the backend automatically routes it to the gateway that owns `cib`.

- If no `payment_method` is provided, the **default gateway** handles the checkout.
- If a gateway is **disabled**, its assigned methods are excluded from routing (but remain in the database for re-enabling).
- The routing map is visible in `GET /super-admin/payment-gateways` → `payment_method_routing`.